

HeapSort

Aula 5

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos

Conteúdo da aula

- 1 Motivação
- 2 Heaps
- 3 Manutenção da propriedade max-heap
- 4 Construção de um max-heap
- 5 HeapSort
- 6 Listas de prioridades
- 7 Exercícios

Introdução

- ▶ como o MERGESORT, o HEAPSORT tem tempo de execução $O(n \lg n)$
- ▶ como o INSERTIONSORT, o HEAPSORT não necessita de um vetor auxiliar
- ▶ o HEAPSORT introduz outra técnica de projeto de algoritmos: usa uma estrutura de dados chamada *heap* para gerenciar informações
- ▶ *heaps* são usados em muitas outras aplicações, tais como listas de prioridades (escalonamento de processos em um computador, simulação de uma lista de eventos, etc), coleta de lixo, entre outras

Definição

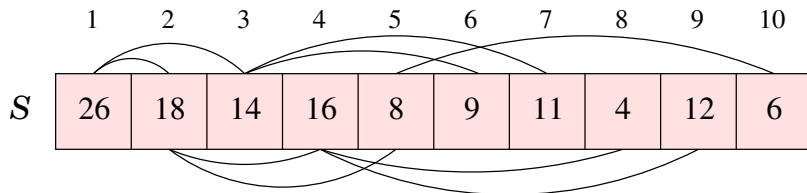
Um **heap** é uma coleção de elementos identificados por suas prioridades, armazenadas em um vetor numérico S , satisfazendo a seguinte propriedade:

$$S[\lfloor i/2 \rfloor] \geq S[i], \quad (1)$$

para todo $i > 1$.

- ▶ um vetor S que representa um *heap* é um objeto com dois atributos: $S.length$ e $S.heap-size$, com $0 \leq S.heap-size \leq S.length$
- ▶ a propriedade (1) é chamada **propriedade max-heap**
- ▶ um vetor S com a propriedade (1) é chamado um **max-heap**

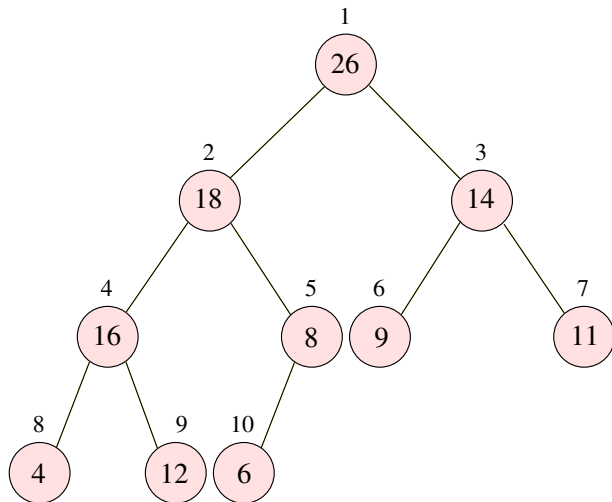
Visualização de um heap



Heaps como árvores binárias

- ▶ é mais interessante ver um max-heap como uma árvore binária
- ▶ isso nos permite verificar a propriedade (1) mais facilmente
- ▶ o conteúdo de um vértice da árvore é maior ou igual aos conteúdos dos vértices que são seus filhos

Heaps como árvores binárias



Altura de um heap

- ▶ olhando para um *heap* como uma árvore, definimos a **altura** de um vértice no *heap* como o número de arestas no caminho mais longo deste vértice até uma folha
- ▶ a altura do *heap* é a altura de sua raiz
- ▶ como um *heap* de n elementos é baseado em uma árvore binária completa, sua altura é $\Theta(\lg n)$
- ▶ as operações básicas sobre *heaps* têm tempo máximo proporcional à altura da árvore e assim têm tempo $O(\lg n)$

Operações básicas

- ▶ vértice pai

$P_{AI}(i)$
1. **devolva** $\lfloor i/2 \rfloor$

- ▶ filho esquerdo

$ESQ(i)$
1. **devolva** $2i$

- ▶ filho direito

$DIR(i)$
1. **devolva** $2i + 1$

Propriedade (max-)heap

- ▶ a propriedade max-heap (1) pode ser reescrita:

$$S[\text{PAI}(i)] \geq S[i], \quad (2)$$

para todo i , com $i > 1$.

Problema

Seja um vetor de números inteiros S com $n > 0$ elementos e um índice i . Se S é visto como uma árvore binária, estabeleça a propriedade max-heap (2) para a subárvore de S com raiz $S[i]$, supondo que as subárvores esquerda e direita do vértice i de S são max-heaps.

Algoritmo DESCE

```

DESCE( $S, i$ )
01.   $e \leftarrow \text{Esq}(i)$ 
02.   $d \leftarrow \text{Dir}(i)$ 
03.  se  $e \leq S.\text{heap-size}$  E  $S[e] > S[i]$ 
04.       $\text{maior} \leftarrow e$ 
05.  senão
06.       $\text{maior} \leftarrow i$ 
07.  se  $d \leq S.\text{heap-size}$  E  $S[d] > S[\text{maior}]$ 
08.       $\text{maior} \leftarrow d$ 
09.  se  $\text{maior} \neq i$ 
10.      troque  $S[i]$  com  $S[\text{maior}]$ 
11.      DESCE( $S, \text{maior}$ )
  
```

Ilustração da execução do algoritmo D_{ESCE}

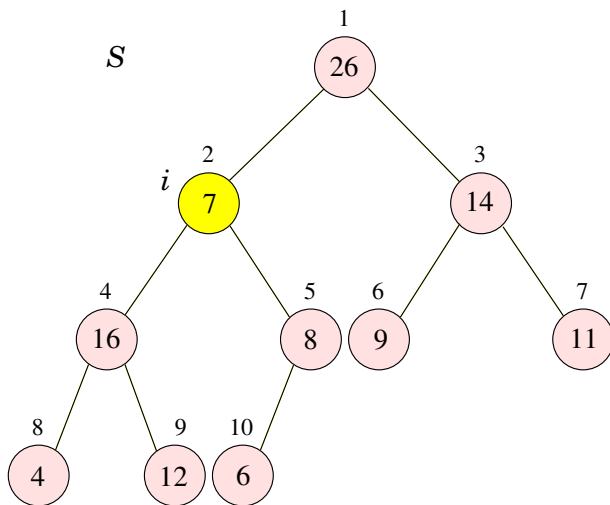


Ilustração da execução do algoritmo D_{ESCE}

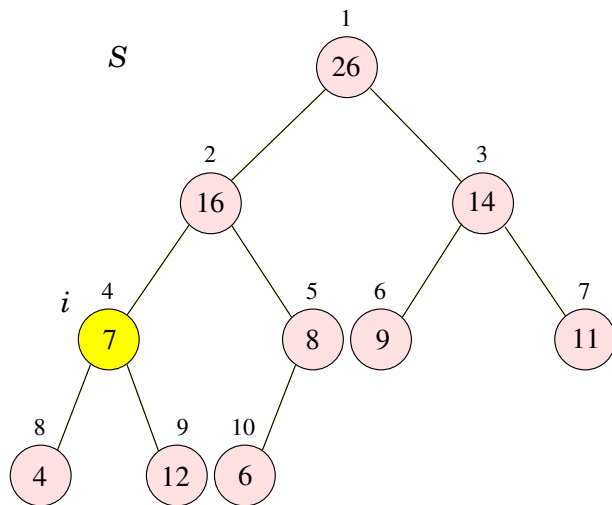
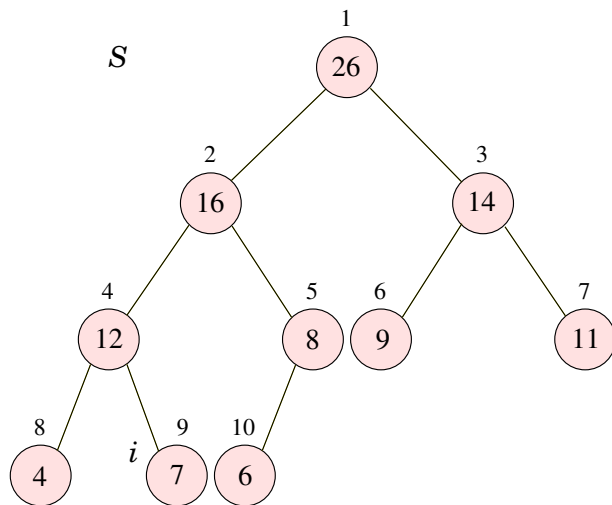


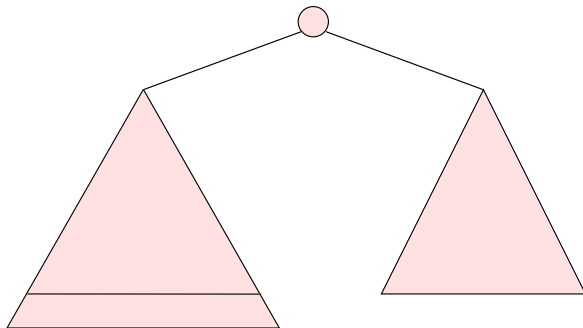
Ilustração da execução do algoritmo D_{ESCE}



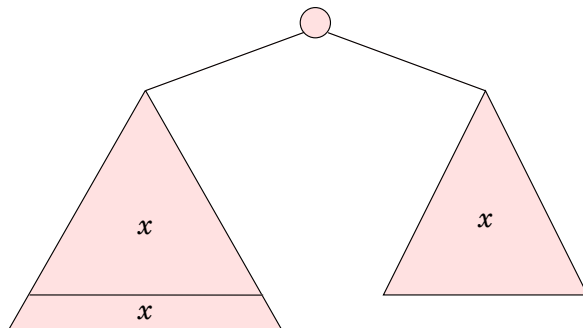
Tempo de execução do algoritmo D_{ESCE}

- ▶ o tempo de execução do algoritmo D_{ESCE} sobre uma subárvore de tamanho n com raiz em um vértice i é dado da seguinte forma:
 - ▶ as linhas 1–8 determinam o maior valor dentre aqueles armazenados em $S[i]$, $S[ESQ(i)]$ e $S[DIR(i)]$ e o tempo gasto é $\Theta(1)$
 - ▶ mais o tempo da chamada recursiva ao algoritmo D_{ESCE} sobre um dos filhos do vértice i , se ocorrer
 - ▶ uma subárvore filha do vértice i tem no máximo $2n/3$ vértices

Proporção de vértices potencialmente eliminados



Proporção de vértices potencialmente eliminados



Tempo de execução do algoritmo D_{ESCE}

- ▶ assim, o tempo de execução de pior caso do algoritmo D_{ESCE} é dado pela recorrência

$$T(n) = T(2n/3) + \Theta(1).$$

- ▶ usando o teorema mestre, temos que $a = 1, b = 3/2$ e $f(n) = c$, para alguma constante $c > 0$. Então, $n^{\log_{3/2} 1} = n^0 = 1$ e assim $f(n) = c = \Theta(1) = \Theta(n^{\log_{3/2} 1}) = \Theta(n^{\log_b a})$. Portanto, usando o caso 2, temos que $T(n) = \Theta(\lg n)$.

Problema

Dado um vetor S de números inteiros com $n > 0$ elementos, transformar S em um max-heap

CONSTRÓI-MAX-HEAP(S)

1. $S.heap-size \leftarrow S.length$
2. **para** $i \leftarrow \lfloor S.length/2 \rfloor$ **até** 1 **passo** -1
3. DESCER(S, i)

Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

	1	2	3	4	5	6	7	8	9	10
<i>S</i>	9	4	8	6	26	16	14	18	12	11

Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

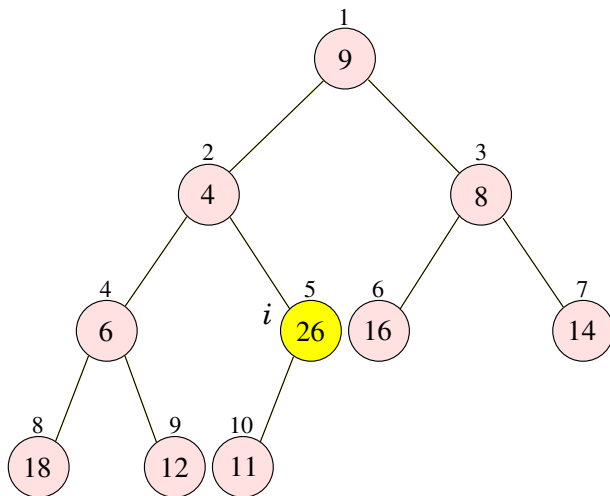


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

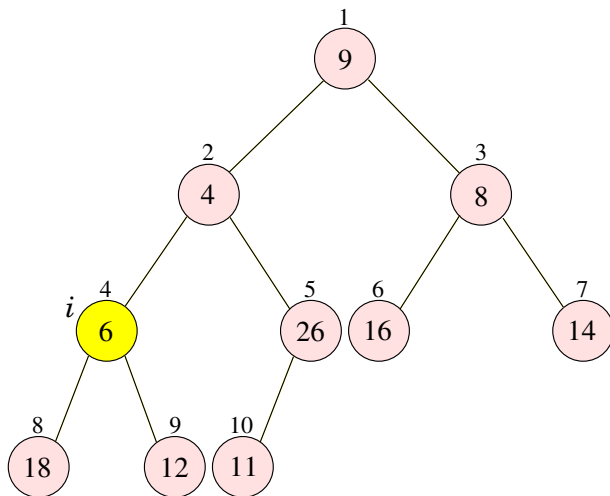


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

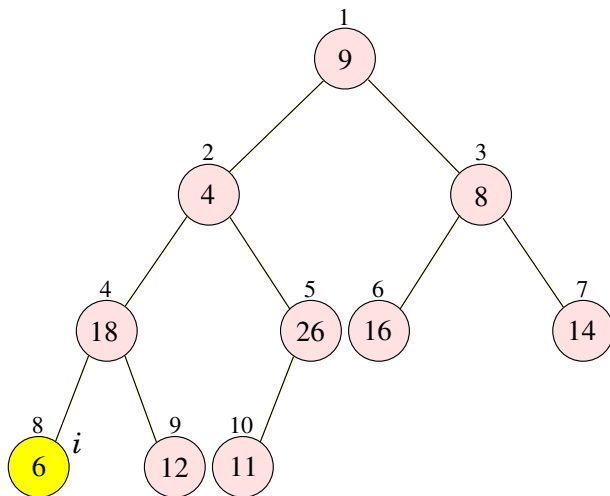


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

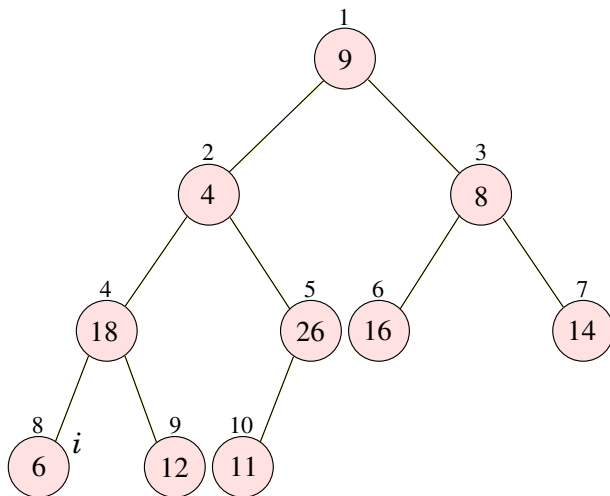


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

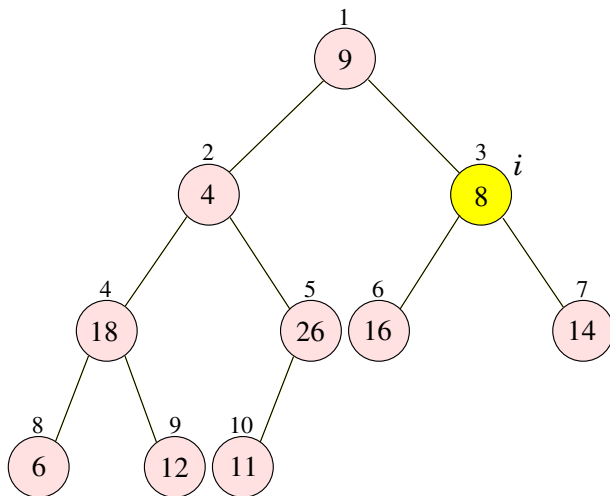


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

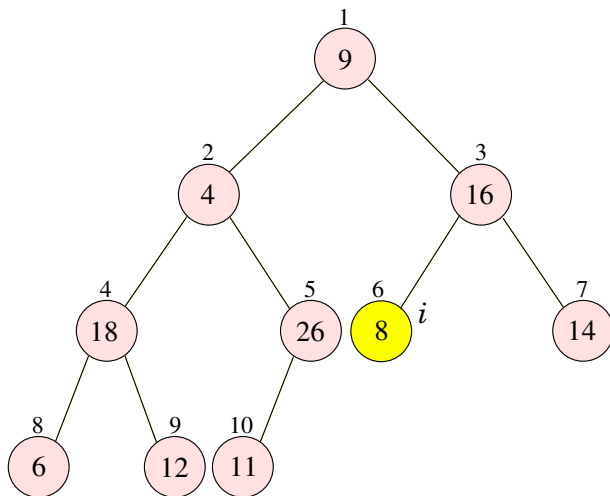


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

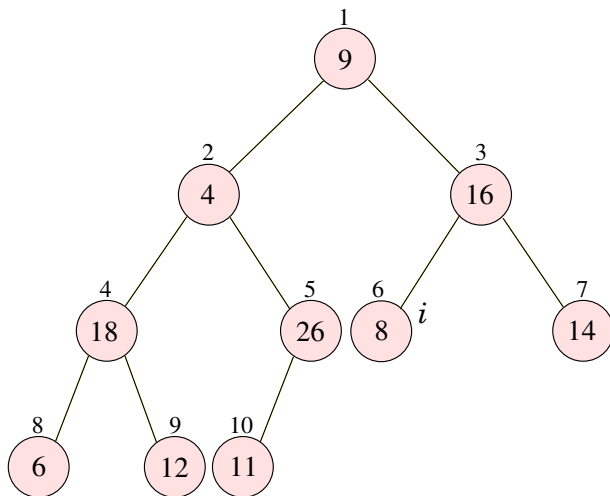


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

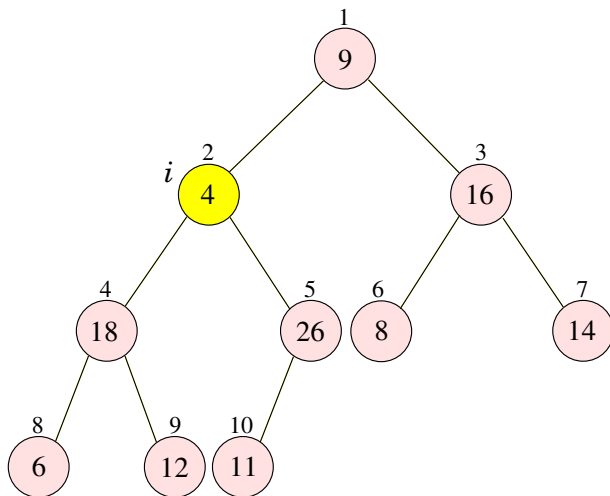


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

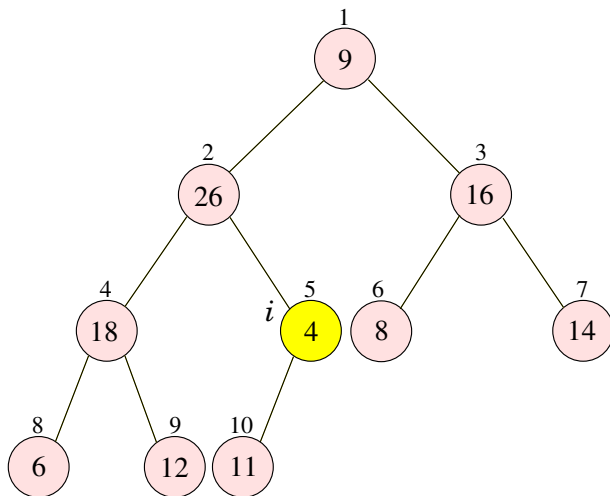


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

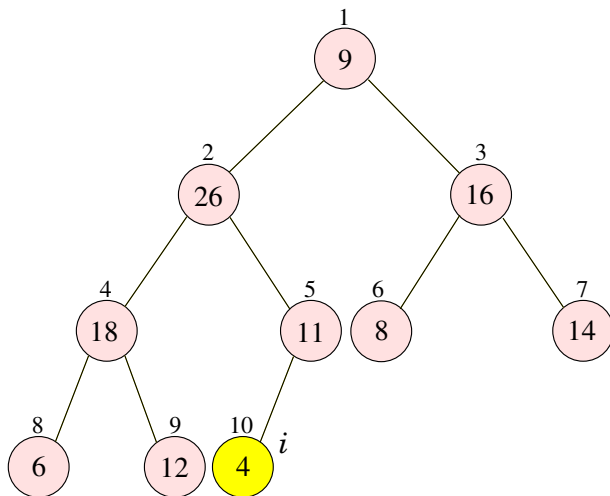


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

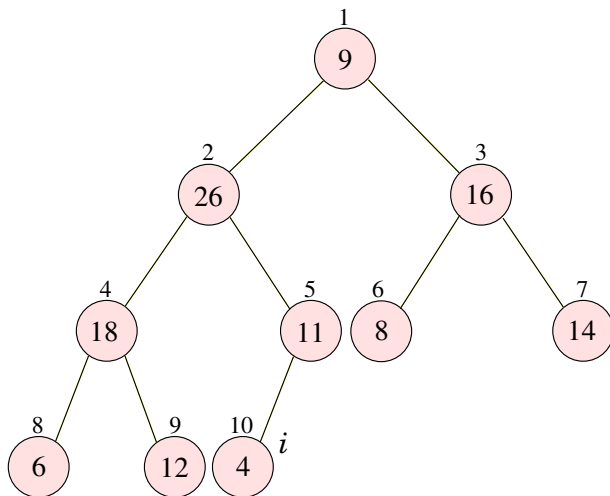


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

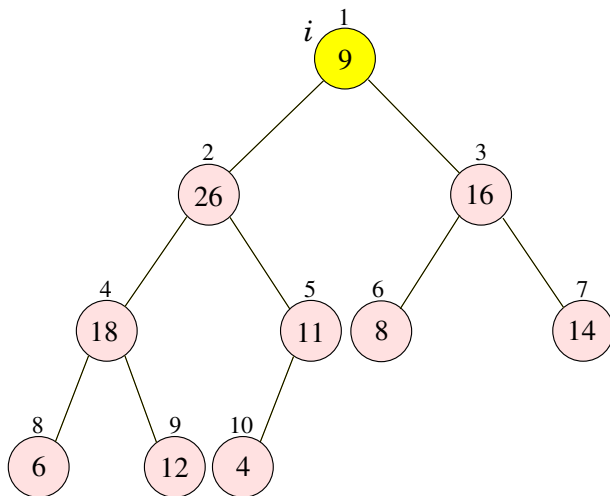


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

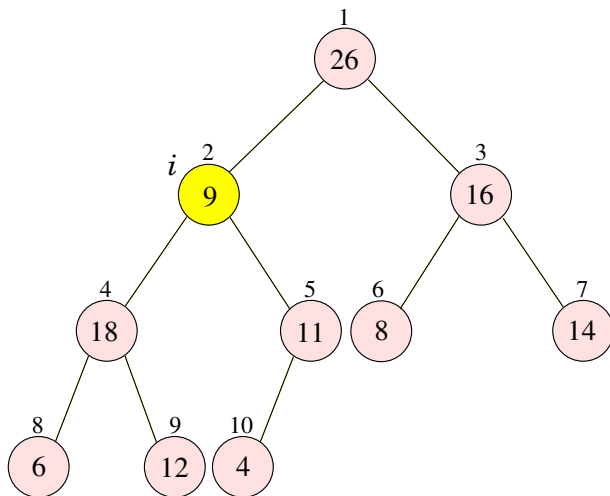


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

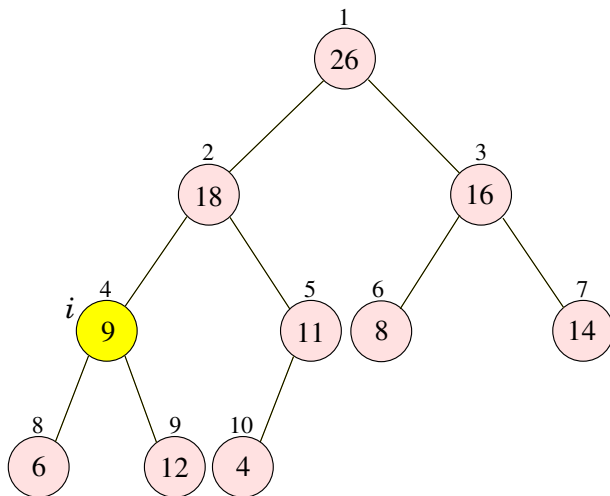


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP

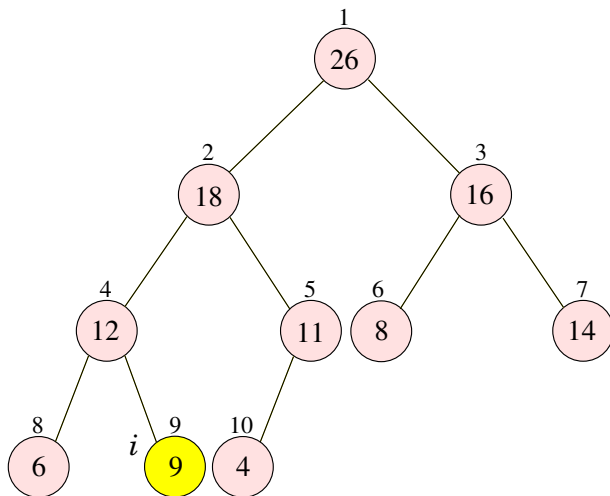
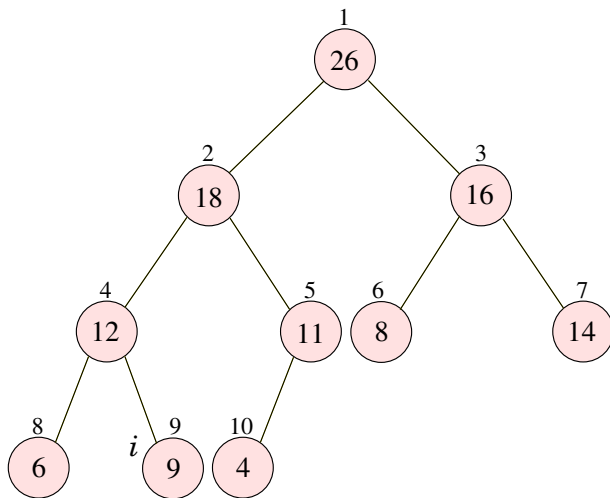


Ilustração da execução do algoritmo

CONSTRÓI MAX HEAP



Correção do algoritmo CONSTRÓI_{MAX}HEAP

Invariante: no início de cada iteração da estrutura de repetição das linhas 2–3, cada vértice $i+1, i+2, \dots, n$ é raiz de um max-heap.

Devemos mostrar que este invariante é verdadeiro antes da primeira iteração da estrutura de repetição, que cada iteração da estrutura de repetição mantém o invariante e que o invariante fornece uma propriedade útil para mostrar a correção quando a estrutura de repetição termina

Correção do algoritmo CONSTRÓI MAX HEAP

Inicialização: Antes da primeira iteração da estrutura de repetição, $i = \lfloor n/2 \rfloor$. Cada vértice $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ é uma folha e, assim, é raiz de uma max-heap trivial.

Manutenção: Pelo invariante, os filhos de um vértice i são raízes de max-heaps. Então, a chamada $\text{DESC}(S, i)$ faz com que o vértice i seja a raiz de um max-heap. Além disso, a chamada a DESC preserva a propriedade que os vértices $i + 1, i + 2, \dots, n$ são raízes de max-heaps. No final da iteração, com o decremento de i , o invariante é reestabelecido para a próxima iteração.

Término: No final, $i = 0$. Pelo invariante, cada vértice $1, 2, \dots, n$ é raiz de um max-heap. Em particular, o vértice 1 é raiz de um max-heap.

Limitante superior para o tempo de execução

- ▶ primeira análise: o algoritmo `DESCE` tem tempo de execução $O(\lg n)$ e é chamado $O(n)$ vezes pelo algoritmo `CONSTRÓIMAXHEAP`; portanto, o tempo de execução do algoritmo `CONSTRÓIMAXHEAP` é $O(n \lg n)$

Limitante superior para o tempo de execução

- análise melhor: observe que um *heap* com n elementos tem altura $\lfloor \lg n \rfloor$ e no máximo $\lceil n/2^{h+1} \rceil$ vértices de altura h ; o tempo gasto pelo algoritmo `DESCE` sobre um vértice de altura h é $O(h)$ e assim, o tempo de execução do algoritmo `CONSTRÓI MAXHEAP` é dado por

$$\begin{aligned}
 \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil ch &\leq \sum_{h=0}^{\infty} \left(\frac{n}{2^{h+1}} \right) ch = cn \sum_{h=0}^{\infty} \frac{h}{2^{h+1}} = cn \left(\frac{1}{2} \right) \sum_{h=0}^{\infty} \frac{h}{2^h} \\
 &= cn \left(\frac{1}{2} \right) \sum_{h=0}^{\infty} h \left(\frac{1}{2} \right)^h \\
 &= cn \left(\frac{1}{2} \right) \left(\frac{\frac{1}{2}}{(1 - \frac{1}{2})^2} \right) \\
 &= cn = O(n).
 \end{aligned}$$

Ordenação com heaps

- ▶ um max-heap pode ser naturalmente usado para descrever um algoritmo de ordenação eficiente
- ▶ algoritmo `HEAPSORT` constrói um max-heap do vetor de entrada e, como um maior elemento do vetor está armazenado na raiz do max-heap, troca esse elemento com o último elemento do vetor, decrementa o tamanho do max-heap e desce o elemento da raiz

Algoritmo HEAPSORT

HEAPSORT(S)

1. CONSTRÓI MAXHEAP(S)
2. **para** $i \leftarrow S.length$ **até** 2 **passo** -1
3. troque $S[1]$ com $S[i]$
4. $S.heap-size \leftarrow S.heap-size - 1$
5. DESCER($S, 1$)

Ilustração da execução do algoritmo HEAPSORT

	1	2	3	4	5	6	7	8	9	10
S	9	4	8	6	26	16	14	18	12	11

Ilustração da execução do algoritmo HEAPSORT

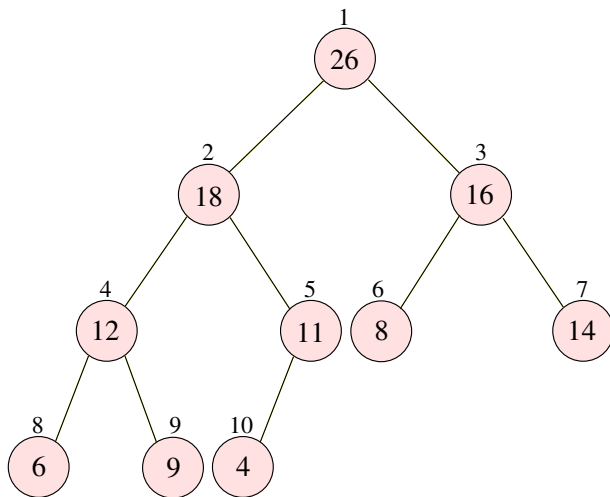


Ilustração da execução do algoritmo HEAPSORT

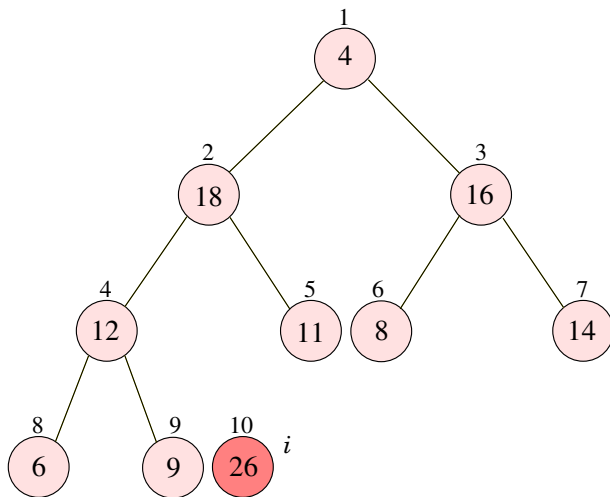


Ilustração da execução do algoritmo HEAPSORT

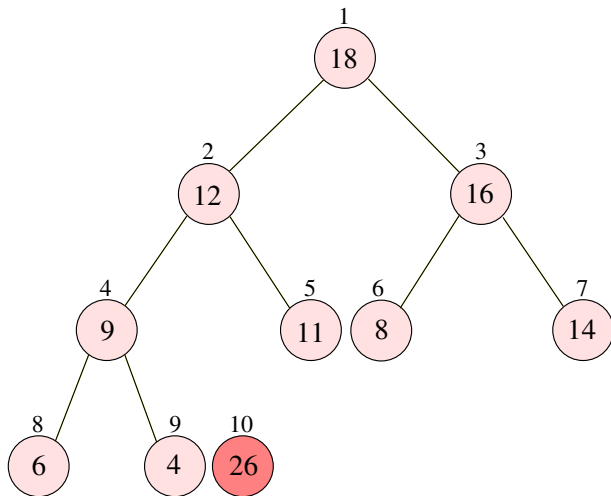


Ilustração da execução do algoritmo HEAPSORT

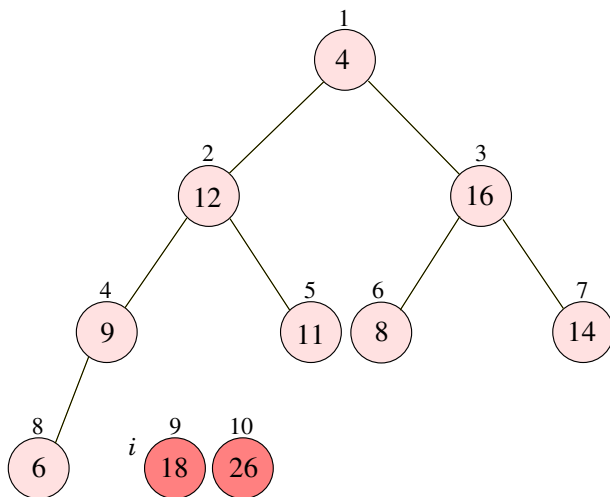


Ilustração da execução do algoritmo HEAPSORT

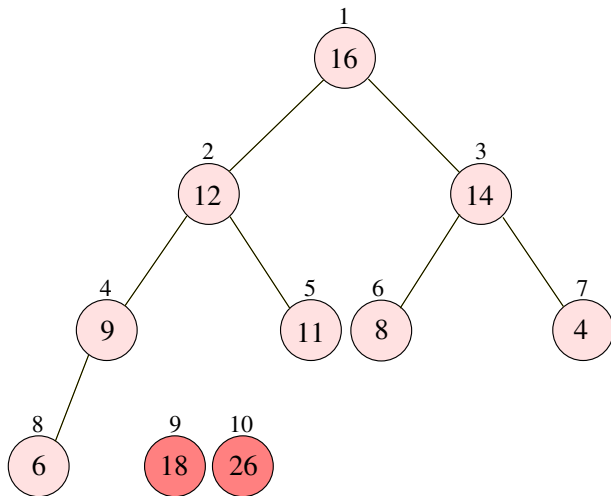


Ilustração da execução do algoritmo HEAPSORT

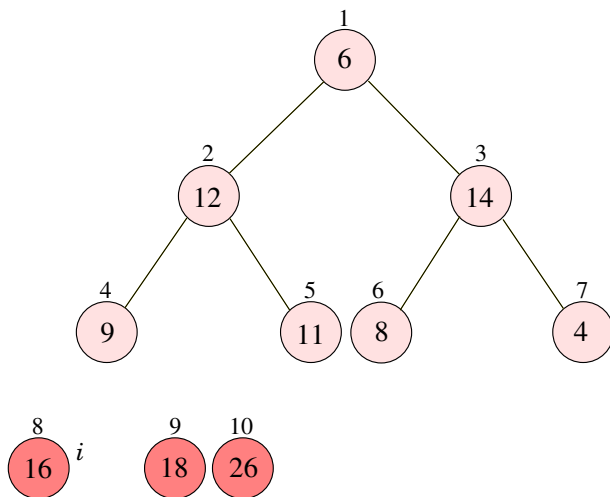


Ilustração da execução do algoritmo HEAPSORT

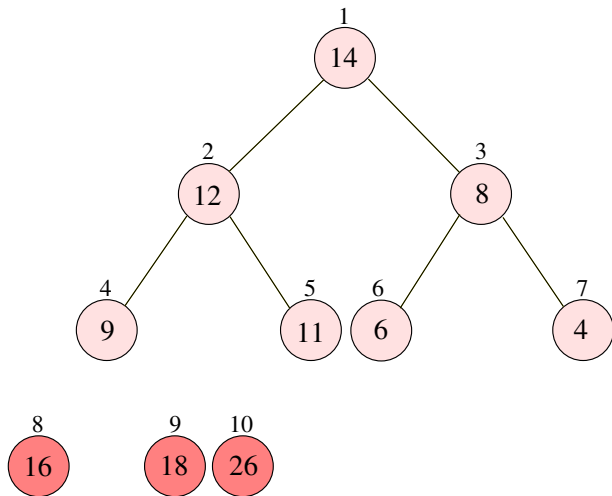


Ilustração da execução do algoritmo HEAPSORT

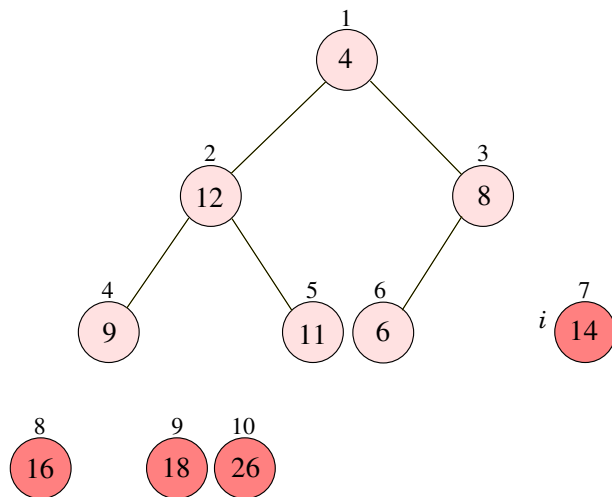


Ilustração da execução do algoritmo HEAPSORT

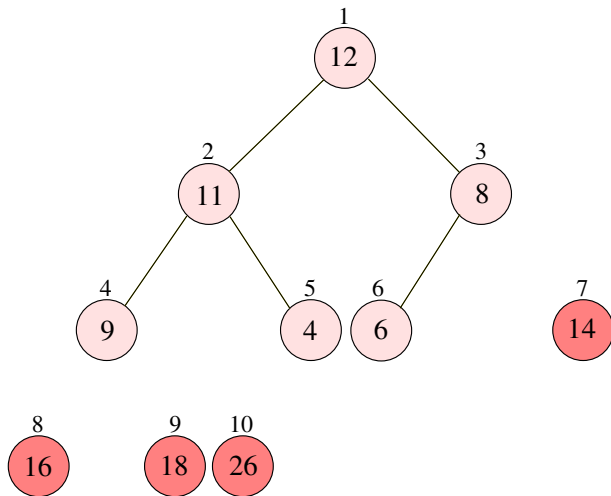


Ilustração da execução do algoritmo HEAPSORT

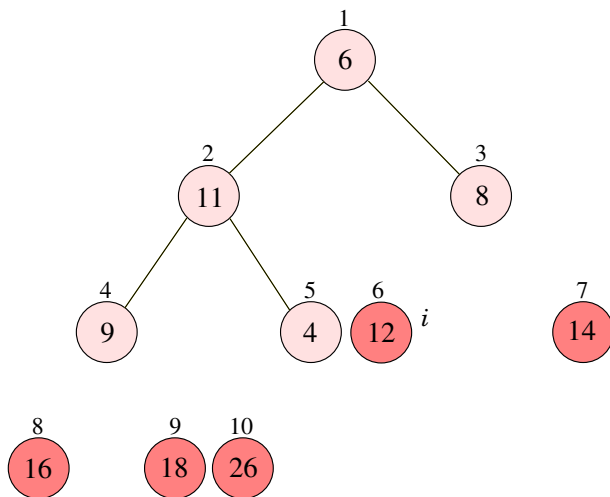


Ilustração da execução do algoritmo HEAPSORT

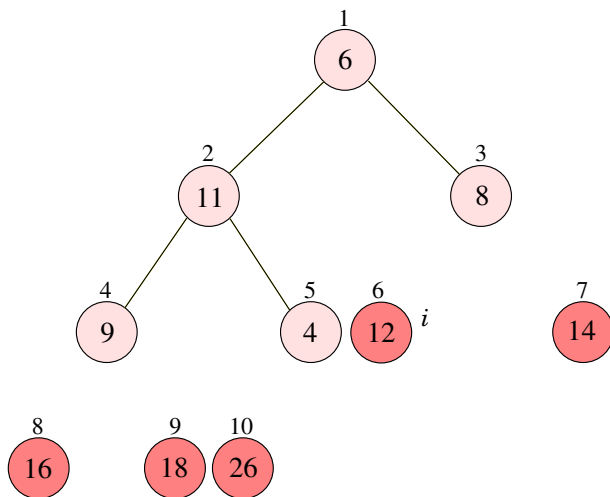


Ilustração da execução do algoritmo HEAPSORT

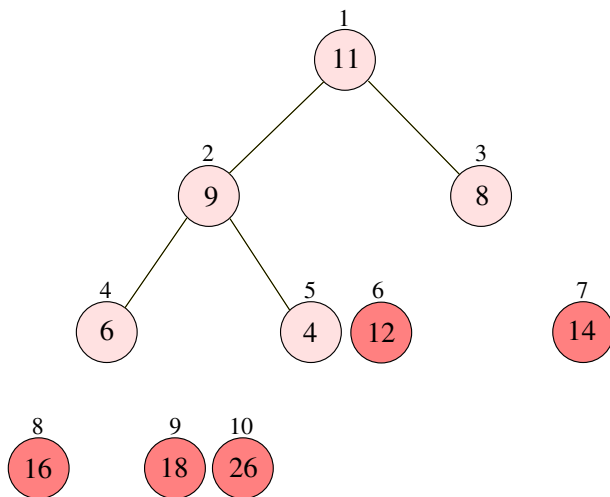


Ilustração da execução do algoritmo HEAPSORT

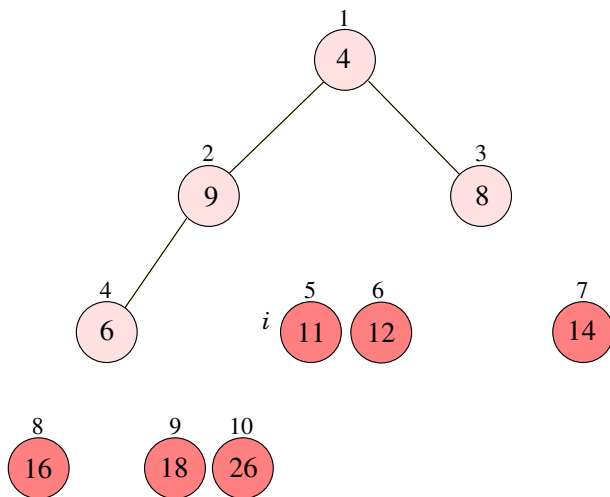


Ilustração da execução do algoritmo HEAPSORT

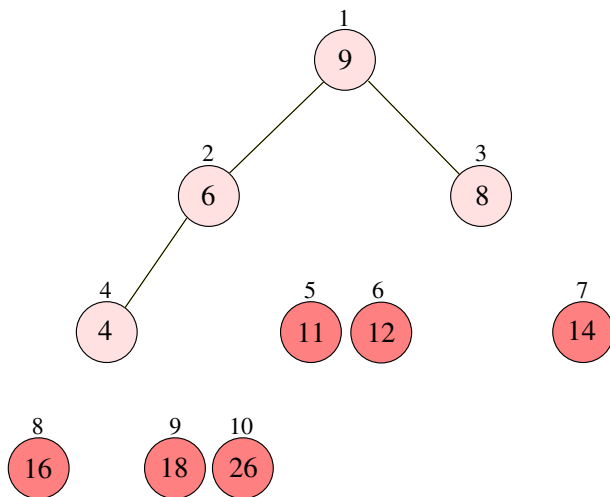


Ilustração da execução do algoritmo HEAPSORT

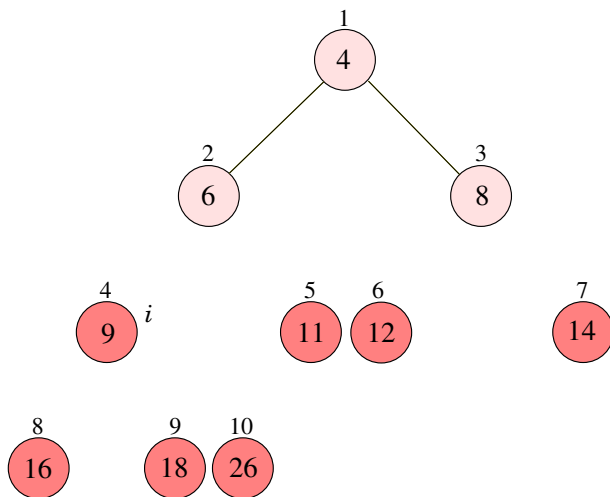


Ilustração da execução do algoritmo HEAPSORT

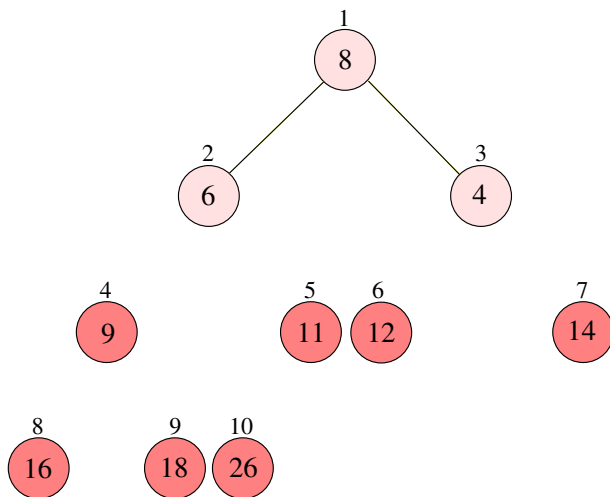


Ilustração da execução do algoritmo HEAPSORT

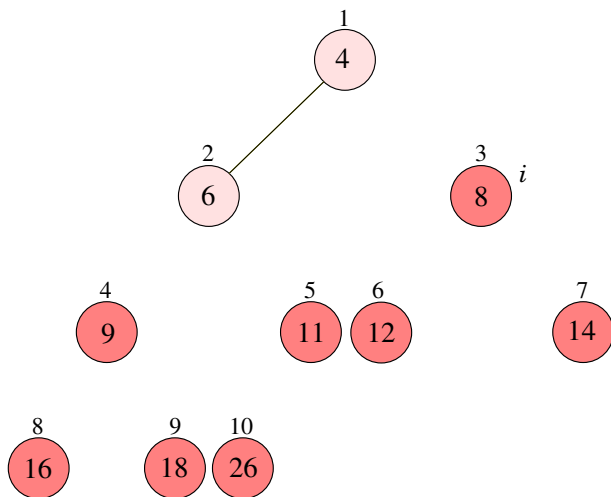


Ilustração da execução do algoritmo HEAPSORT

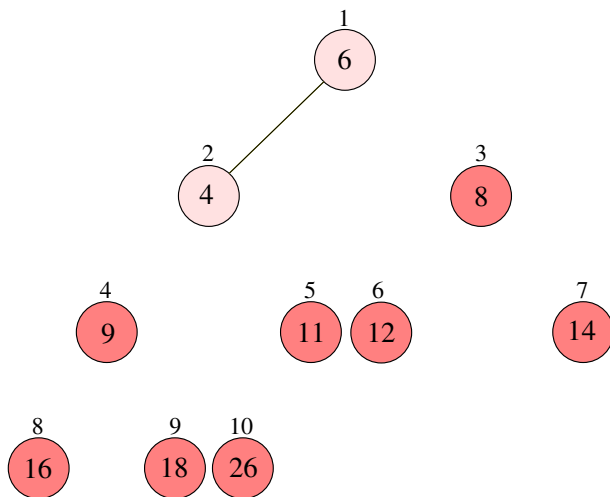


Ilustração da execução do algoritmo HEAPSORT

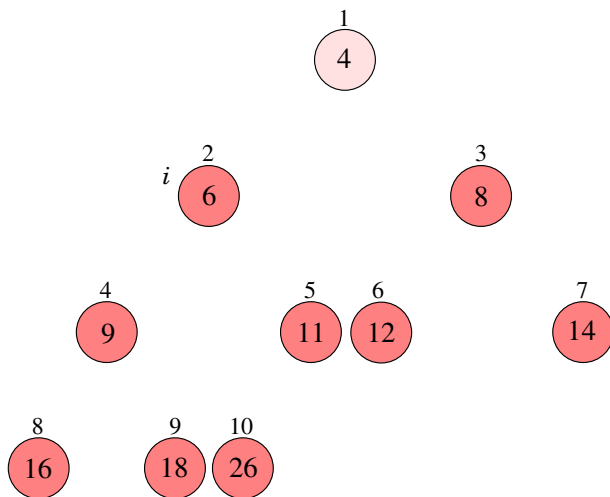


Ilustração da execução do algoritmo HEAPSORT

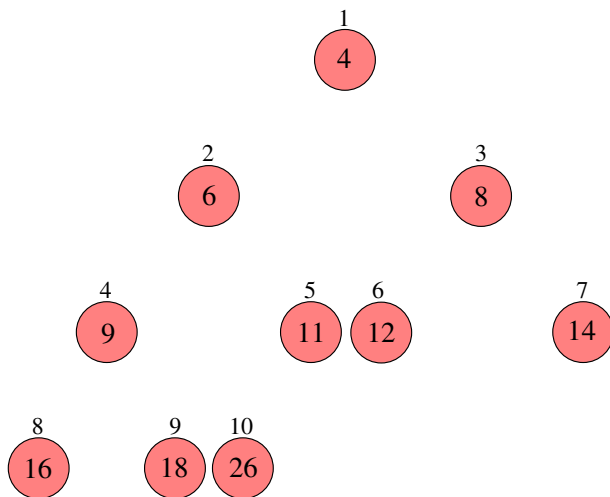


Ilustração da execução do algoritmo HEAPSORT

	1	2	3	4	5	6	7	8	9	10
<i>S</i>	4	6	8	9	11	12	14	16	18	26

Tempo de execução do HEAPSORT

- ▶ O tempo de execução do HEAPSORT é $O(n \lg n)$ já que a chamada ao algoritmo CONSTRÓI MAXHEAP tem tempo de execução $O(n)$ e cada uma das $n - 1$ chamadas ao algoritmo DESCE tem tempo de execução $O(\lg n)$ sobre um vetor de entrada de n elementos

Motivação

- ▶ ocorrem em muitas aplicações: escalonamento de tarefas em um computador, simulador de eventos, etc.
- ▶ listas de max-prioridades e listas de min-prioridades
- ▶ uma **lista de max-prioridades** é uma estrutura de dados para manutenção de um conjunto de elementos S , onde a cada elemento está associada uma prioridade
- ▶ as seguintes operações são associadas a uma lista de max-prioridades:
 - (1) inserção de um elemento no conjunto S ;
 - (2) consulta da maior prioridade em S ;
 - (3) remoção do elemento de maior prioridade em S ;
 - (4) aumento da prioridade de um elemento de S .
- ▶ um max-heap pode ser usado para implementar uma lista de max-prioridades

Operações básicas

- ▶ operação (2): consulta a maior prioridade em S

```
MAIORPRIORIDADE( $S$ )  
1.  devolva  $S[1]$ 
```

- ▶ tempo de execução do algoritmo MAIORPRIORIDADE é constante, ou seja, $\Theta(1)$

Operações básicas

- ▶ operação (3): extrai de S o elemento de prioridade máxima

```
EXTRAIMÁXIMA( $S$ )  
1.  se  $S.heap-size < 1$   
2.    erro "heap underflow"  
3.  maior  $\leftarrow S[1]$   
4.   $S[1] \leftarrow S[S.heap-size]$   
5.   $S.heap-size \leftarrow S.heap-size - 1$   
6.  DESCE( $S, 1$ )  
7.  devolva maior
```

- ▶ tempo de execução do algoritmo EXTRAIMÁXIMA é o tempo gasto pelo algoritmo DESCE. Portanto, seu tempo de execução é $O(\lg n)$

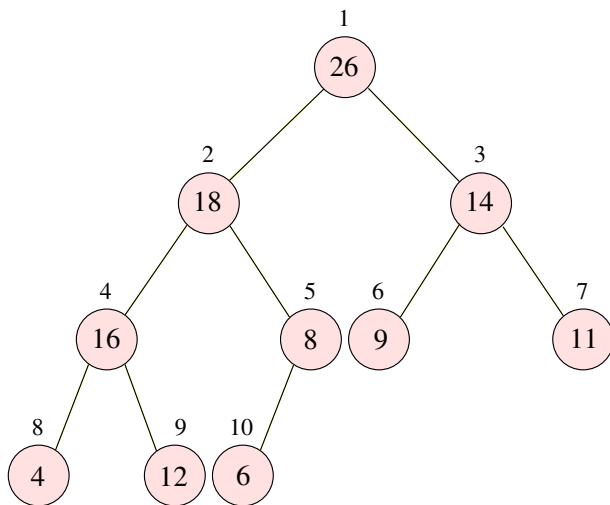
Operações básicas

- ▶ para as operações de inserção de uma nova prioridade e aumento de uma prioridade existente, devemos nos preocupar com os seguintes casos

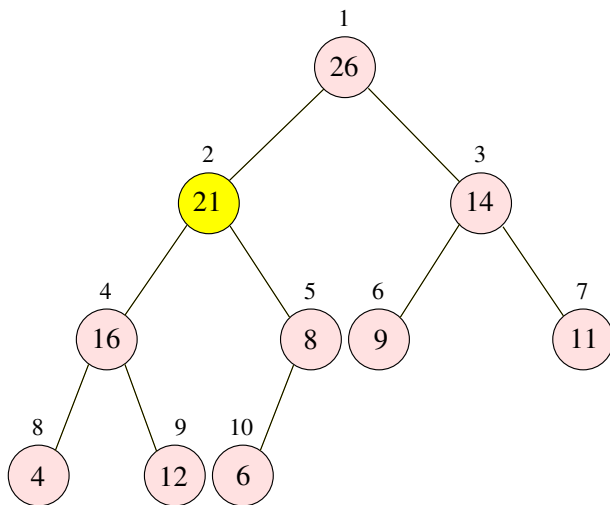
Operações básicas

	1	2	3	4	5	6	7	8	9	10
<i>S</i>	26	18	14	16	8	9	11	4	12	6

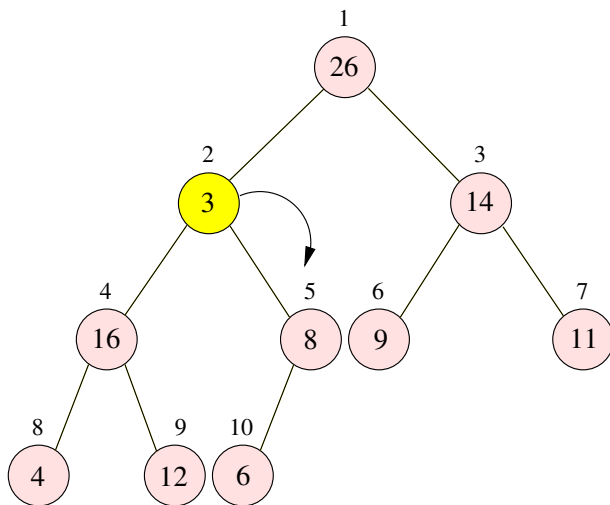
Operações básicas



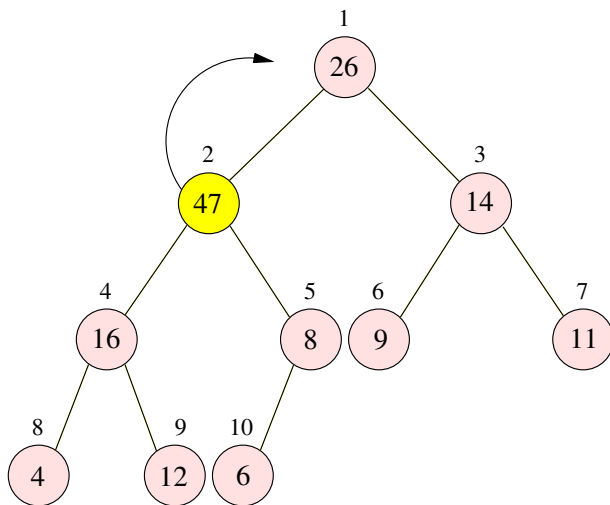
Operações básicas



Operações básicas



Operações básicas

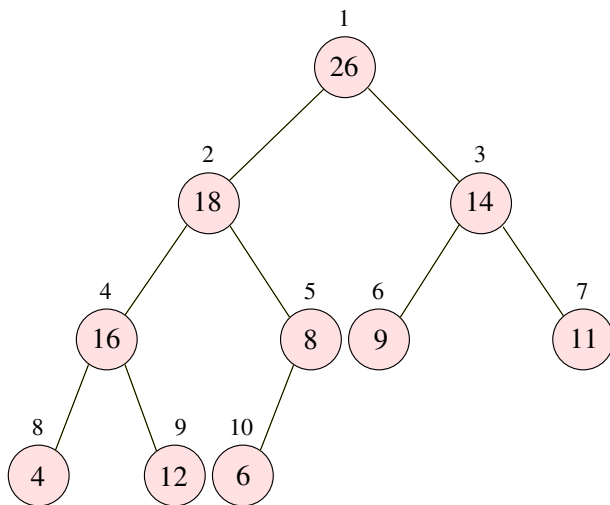


Operações básicas

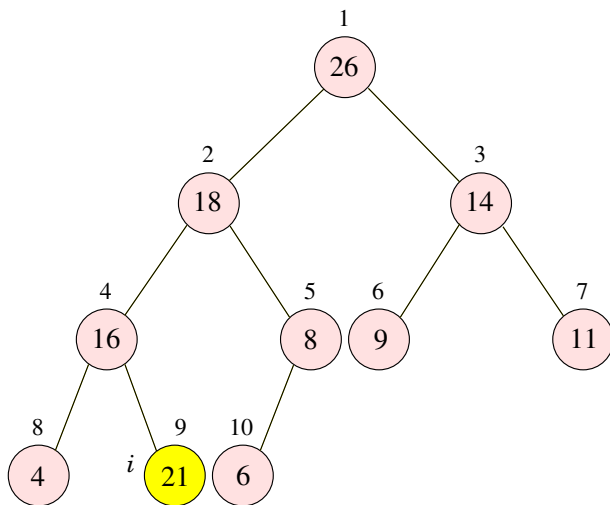
$\text{SOBE}(S, i)$

1. **enquanto** $i > 1$ **E** $S[\text{PAI}(i)] < S[i]$
2. troque $S[\text{PAI}(i)]$ e $S[i]$
3. $i \leftarrow \text{PAI}(i)$

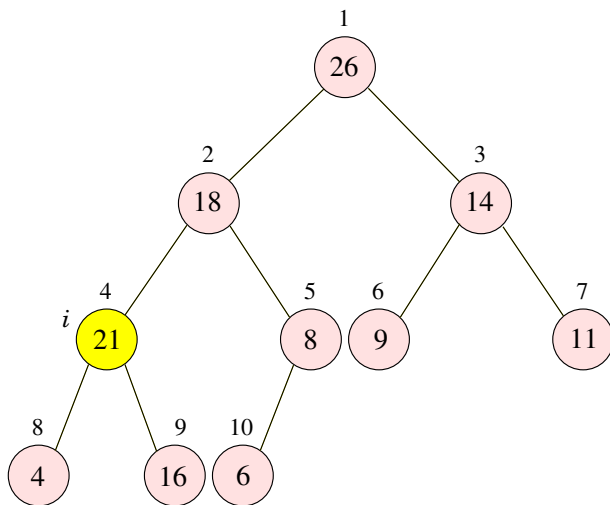
Operações básicas



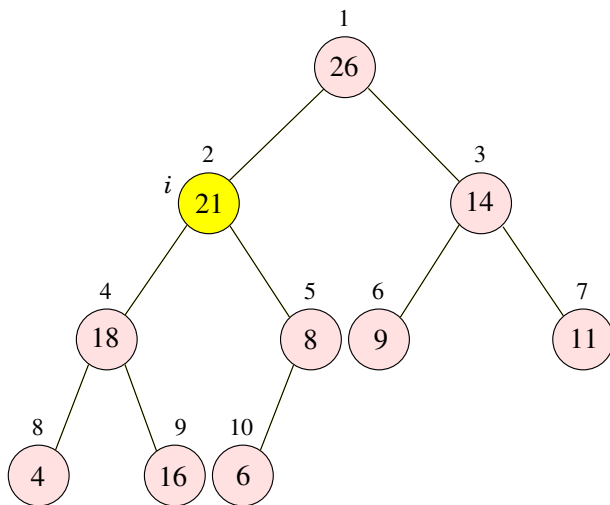
Operações básicas



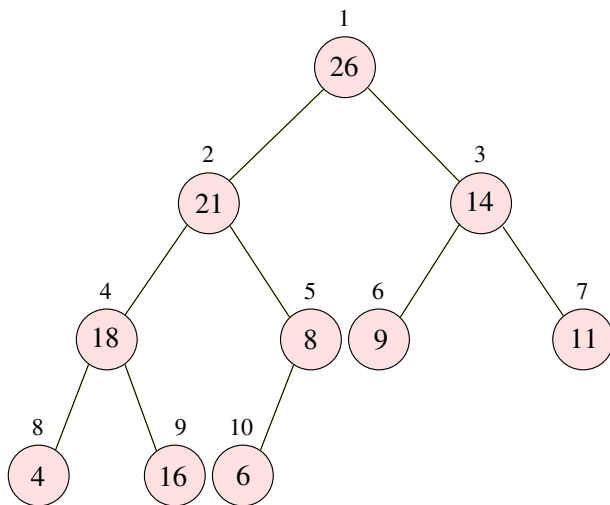
Operações básicas



Operações básicas



Operações básicas



Operações básicas

Tempo de execução do algoritmo S_{OBE} :

- ▶ o tempo de execução de pior caso do algoritmo S_{OBE} é proporcional à altura da árvore binária correspondente à S , isto é, $O(\lg n)$

Operações básicas

- ▶ operação (4): aumento da prioridade de um elemento de S

AUMENTAPRIORIDADE(S, i, p)

1. **se** $p < S[i]$
2. **erro** “nova prioridade é menor que a atual”
3. **senão**
4. $S[i] \leftarrow p$
5. SOBE(S, i)

- ▶ tempo de execução do algoritmo AUMENTAPRIORIDADE é o tempo gasto pela chamada ao algoritmo SOBE e, portanto, é $O(\lg n)$

Operações básicas

- ▶ operação (1): inserção de um elemento no conjunto S

INSERELISTAMAX(S, p)

1. $S.heap-size \leftarrow S.heap-size + 1$
2. $S[S.heap-size] \leftarrow p$
3. SOBE($S, S.heap-size$)

- ▶ tempo de execução do algoritmo INSERELISTAMAX é o tempo gasto pela chamada ao algoritmo SOBE e, portanto, é $O(\lg n)$

Exercícios

- 6.1-1 Quais são os números mínimo e máximo de elementos em um max-heap de altura h ?
- 6.1-2 Mostre que um *heap* com n elementos tem altura $\lceil \lg n \rceil$.
- 6.1-3 Mostre que em qualquer subárvore de um max-heap, a raiz da subárvore contém o maior valor de todos aqueles que ocorrem naquela subárvore.
- 6.1-4 Em um max-heap, onde pode estar armazenado o elemento de menor prioridade, considerando que todos os elementos são distintos?
- 6.1-5 Um vetor arranjado em ordem crescente é um min-heap?
- 6.1-6 A sequência $\langle 23, 17, 14, 6, 13, 10, 1, 5, 7, 12 \rangle$ é um max-heap?
- 6.1-7 Mostre que com a representação de um *heap* de n elementos em um vetor, as folhas do *heap* estão armazenadas nos compartimentos do vetor de índices $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$.

Exercícios

- 6.2-1 Ilustre a operação de $\text{DESCE}(S, 3)$ sobre o vetor $S = \langle 27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0 \rangle$.
- 6.2-2 Suponha que você queira manter um min-heap. Escreva uma função equivalente à função DESCE para um max-heap, que mantém a propriedade min-heap.
- 6.2-3 Qual o efeito de chamar $\text{DESCE}(S, i)$ quando a prioridade $S[i]$ é maior que as prioridades de seus filhos?
- 6.2-4 Qual o efeito de chamar $\text{DESCE}(S, i)$ para $i > \lfloor S.\text{heap-size}/2 \rfloor$?

Exercícios

- 6.2-5 O código da função `DESCE` é muito eficiente em termos de fatores constantes, exceto possivelmente pela chamada recursiva que pode fazer com que alguns compiladores produzam um código ineficiente. Escreva uma função não-recursiva eficiente equivalente à função `DESCE`.
- 6.2-6 Sem usar o teorema mestre, mostre que o tempo de execução de pior caso do algoritmo `DESCE` sobre um *heap* de tamanho n é $\Omega(\lg n)$. *Sugestão:* para um heap com n elementos, forneça valores aos vértices que façam o algoritmo `DESCE` ser chamado recursivamente para todo vértice em um caminho da raiz até uma folha.

Exercícios

- 6.3-1 Ilustre a operação da função `CONSTRÓIMAXHEAP` sobre o vetor $S = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$.
- 6.3-2 Por que fazemos com que a estrutura de repetição da função `CONSTRÓIMAXHEAP` controlada por i seja decrescente de $\lfloor S.length/2 \rfloor$ até 1 ao invés de crescente de 1 até $\lfloor S.length/2 \rfloor$?
- 6.3-3 Mostre que existem no máximo $\lceil n/2^{h+1} \rceil$ vértices de altura h em um *heap* com n elementos.

Exercícios

- 6.4-1 Ilustre a execução da função `HEAPSORT` sobre o vetor $S = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$.
- 6.4-2 Mostre que o algoritmo `HEAPSORT` está correto usando o seguinte invariante do processo iterativo:
- No início de cada iteração da estrutura de repetição da função `HEAPSORT`, o vetor $S[1..i]$ é um max-heap contendo os i menores elementos de $S[1..n]$ e o vetor $S[i+1..n]$ contém os $n-i$ maiores elementos de $S[1..n]$ em ordem crescente.
- 6.4-3 Qual o tempo de execução do `HEAPSORT` sobre um vetor S de tamanho n que já está arranjado em ordem crescente? E em ordem decrescente?
- 6.4-4 Mostre que o tempo de execução de pior caso do `HEAPSORT` é $\Omega(n \lg n)$.

Exercícios

- 6.5-1 Ilustre a operação do algoritmo `EXTRAIMÁXIMA` sobre o vetor $S = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.
- 6.5-2 Ilustre a operação do algoritmo `INSERELISTA(S, 10)` sobre a lista de prioridades $S = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$.
- 6.5-3 Escreva pseudocódigos para os processos que implementam as operações `MENORPRIORIDADE`, `EXTRAIMÍNIMO`, `DIMINUIPRIORIDADE` e `INSERELISTAMIN`. Esses algoritmos devem implementar uma lista de min-prioridades com um min-heap.
- 6.5-8 A operação `REMOVELISTAMAX(S, i)` remove a prioridade do vértice i de uma lista de max-prioridades S . Escreva um algoritmo para `REMOVELISTAMAX` com tempo de execução $O(\lg n)$ para um max-heap com n elementos.

Problemas

6-1 Construção de um *heap* usando inserção

Podemos construir um *heap* chamando repetidamente o algoritmo `INSERELISTAMAX` para inserir elementos no *heap*.

Considere a seguinte variação no algoritmo `CONSTRÓIMAXHEAP`:

`CONSTRÓIMAXHEAP'(S)`

1. $S.\text{heap-size} \leftarrow 1$
2. **para** $i \leftarrow 2$ **até** $S.\text{length}$
3. `INSERELISTAMAX(S, S[i])`

- (a) Os algoritmos `CONSTRÓIMAXHEAP` e `CONSTRÓIMAXHEAP'` sempre criam o mesmo *heap* quando são chamados sobre o mesmo vetor de entrada? Prove que eles fazem isso ou mostre um contra-exemplo.
- (b) Mostre que no pior caso o algoritmo `CONSTRÓIMAXHEAP'` requer tempo $\Theta(n \lg n)$ para construir um *heap* de n elementos.

Problemas

6-2 Análise de *heaps* d -ários

Um **heap** d -ário é como um *heap* binário, mas os vértices internos têm d filhos e não 2. A raiz pode ter menos que d filhos apenas no caso em que $n \leq d$.

- (a) Como representar um *heap* d -ário em um vetor?
- (b) Qual é a altura de um *heap* d -ário de n elementos em termos de n e d ?
- (c) Forneça um algoritmo eficiente equivalente ao algoritmo `EXTRAÍMÁXIMA` para um *heap* d -ário. Analise seu tempo de execução em termos de d e n .
- (d) Forneça um algoritmo eficiente equivalente ao algoritmo `INSERELISTAMAX` para um *heap* d -ário. Analise seu tempo de execução em termos de d e n .
- (e) Forneça um algoritmo eficiente equivalente ao algoritmo `AUMENTAPRIORIDADE` para um *heap* d -ário. Analise seu tempo de execução em termos de d e n .